

Society Management System

Complete Implementation Blueprint

Folder structure • Database schema & relationships • Module-to-code mapping • Redis usage plan • REST API design • RBAC matrix

Prepared by	Anshil
Prepared for	Creatuares — Backend Developer Internship Assessment
Recommended stack	Node.js, Express.js, PostgreSQL, Redis
Document date	July 2026

Table of Contents

1	How to Use This Document
2	Recommended Tech Stack
3	Complete Folder Structure (Backend + Frontend)
4	Database Design — Tables, Columns & Relationships
5	Entity Relationship Summary
6	Redis Usage Plan — What, Where & Why
7	Functional Module → Code Mapping
8	REST API Endpoint Design
9	Role-Based Access Control (RBAC) Matrix
10	Security & Validation Checklist
11	Testing & Documentation Plan
12	Submission Checklist (Don't Forget)

1. How to Use This Document

This document is a single reference you can build the entire Creatuares "Society Management System" assessment from, end to end. It is organised so that each piece answers one practical question you will hit while coding: what folders do I create, what tables do I need and how are they related, where exactly does each functional module's code live, where does Redis actually fit in, what API routes should exist, and what does the evaluator expect to see.

Treat Section 4 (Database Design) and Section 3 (Folder Structure) as the backbone -- build those first. Then use Section 7 (Module → Code Mapping) as your day-by-day task list, module by module. Section 6 (Redis) is intentionally kept separate and optional-first: the project works correctly on PostgreSQL alone, and Redis is layered on top purely for performance, scalability and security hardening -- exactly the kind of addition that makes a submission stand out in the evaluation criteria ("Code Quality", "Security Practices", "Overall Implementation").

2. Recommended Tech Stack

Layer	Choice	Why
Backend runtime	Node.js (v20 LTS) + Express.js	Matches the assessment's REST API expectation; fast to build, well documented, huge middleware ecosystem.
Primary database	PostgreSQL 15+	Relational data (residents, flats, bills, payments) needs strong normalization + foreign keys + transactions -- a perfect fit, and explicitly rewarded under "Database Design" in the evaluation criteria.
ORM / Query layer	Prisma or Sequelize (Knex if you prefer raw SQL control)	Migrations, schema-as-code, and protection against SQL injection out of the box.
Cache / Session / Queue	Redis 7+	Token/session store, rate limiting, caching hot reads, and backing BullMQ job queues (see Section 6).
Authentication	JWT (access + refresh token pair) + bcrypt	Explicitly requested ("JWT or equivalent") -- stateless access tokens, rotated refresh tokens stored in Redis.
File uploads	Multer (local disk in dev) then Cloudinary / AWS S3 in production	Needed for complaint image uploads; keeping storage abstracted lets you swap local to cloud without touching business logic.
Validation	Zod or Joi	Centralised request validation before it ever reaches a controller.
Job queue	BullMQ (Redis-backed)	Monthly bill generation, notification dispatch, image resizing -- all async, all off the request/response cycle.
API docs	Swagger / OpenAPI (swagger-jsdoc + swagger-ui-express) + Postman collection export	Directly requested as a submission artifact.
Testing	Jest + Supertest	Unit tests for services, integration tests for routes.
Frontend	React.js (Vite) + Axios + React Router	Role-based dashboards for Admin / Committee / Resident.
Deployment (optional)	Render or Railway (API) + Vercel/Netlify (frontend) + Supabase/Neon (Postgres) + Upstash (Redis)	All have generous free tiers -- good for an optional live deployment link.

3. Complete Folder Structure

3.1 Backend (Node.js + Express + PostgreSQL + Redis)

```
society-management-system/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   ├── db.config.js           # PostgreSQL pool / Prisma client
│   │   │   ├── redis.config.js       # Redis client (ioredis)
│   │   │   ├── jwt.config.js         # token secrets, expiry values
│   │   │   ├── multer.config.js      # upload storage + file filters
│   │   │   └── env.js                # centralised env var loader
│   │   ├── models/                  # Sequelize/Prisma models or SQL modules
│   │   │   ├── role.model.js
│   │   │   ├── society.model.js
│   │   │   ├── user.model.js
│   │   │   ├── block.model.js
│   │   │   ├── flat.model.js
│   │   │   ├── residentProfile.model.js
│   │   │   ├── committeeMember.model.js
│   │   │   ├── maintenanceBill.model.js
│   │   │   ├── payment.model.js
│   │   │   ├── complaint.model.js
│   │   │   ├── complaintImage.model.js
│   │   │   ├── complaintStatusLog.model.js
│   │   │   ├── serviceRequest.model.js
│   │   │   ├── announcement.model.js
│   │   │   ├── auditLog.model.js
│   │   │   └── notification.model.js
│   │   ├── migrations/              # timestamped schema migrations
│   │   ├── seeders/                 # roles seed, demo admin, demo society
│   │   └── controllers/
│   │       ├── auth.controller.js
│   │       ├── user.controller.js
│   │       ├── resident.controller.js
│   │       ├── committee.controller.js
│   │       ├── flat.controller.js
│   │       ├── maintenanceBill.controller.js
│   │       ├── payment.controller.js
│   │       ├── complaint.controller.js
│   │       ├── serviceRequest.controller.js
│   │       ├── announcement.controller.js
│   │       ├── dashboard.controller.js
│   │       └── auditLog.controller.js
│   └── services/                    # business logic, kept out of controllers
```

```

■ ■ ■ ■■■ auth.service.js
■ ■ ■ ■■■ user.service.js
■ ■ ■ ■■■ maintenanceBill.service.js
■ ■ ■ ■■■ payment.service.js
■ ■ ■ ■■■ complaint.service.js
■ ■ ■ ■■■ serviceRequest.service.js
■ ■ ■ ■■■ announcement.service.js
■ ■ ■ ■■■ notification.service.js
■ ■ ■ ■■■ auditLog.service.js
■ ■ ■ ■■■ cache.service.js      # thin Redis get/set/del wrapper
■ ■ ■
■ ■ ■■■ routes/
■ ■ ■ ■■■ index.js             # mounts every router below on /api
■ ■ ■ ■■■ auth.routes.js
■ ■ ■ ■■■ user.routes.js
■ ■ ■ ■■■ resident.routes.js
■ ■ ■ ■■■ committee.routes.js
■ ■ ■ ■■■ flat.routes.js
■ ■ ■ ■■■ maintenanceBill.routes.js
■ ■ ■ ■■■ payment.routes.js
■ ■ ■ ■■■ complaint.routes.js
■ ■ ■ ■■■ serviceRequest.routes.js
■ ■ ■ ■■■ announcement.routes.js
■ ■ ■ ■■■ dashboard.routes.js
■ ■ ■ ■■■ auditLog.routes.js
■ ■ ■
■ ■ ■■■ middlewares/
■ ■ ■ ■■■ auth.middleware.js    # verify JWT, attach req.user
■ ■ ■ ■■■ rbac.middleware.js    # allowRoles('admin','committee')
■ ■ ■ ■■■ validate.middleware.js # runs Joi/Zod schema on req body
■ ■ ■ ■■■ upload.middleware.js   # multer instance for complaint images
■ ■ ■ ■■■ rateLimiter.middleware.js # Redis-backed rate limiting
■ ■ ■ ■■■ auditLogger.middleware.js # writes audit_logs on mutations
■ ■ ■ ■■■ errorHandler.middleware.js
■ ■ ■
■ ■ ■■■ validators/
■ ■ ■ ■■■ auth.validator.js
■ ■ ■ ■■■ complaint.validator.js
■ ■ ■ ■■■ serviceRequest.validator.js
■ ■ ■ ■■■ maintenanceBill.validator.js
■ ■ ■ ■■■ announcement.validator.js
■ ■ ■
■ ■ ■■■ utils/
■ ■ ■ ■■■ apiResponse.js        # {success, data, message} envelope
■ ■ ■ ■■■ asyncHandler.js       # try/catch wrapper for controllers
■ ■ ■ ■■■ logger.js             # winston logger

```

```

■ ■ ■ ■■■ generateTokens.js      # sign/verify access+refresh JWT
■ ■ ■ ■■■ paginate.js           # shared pagination helper
■ ■ ■ ■■■ fileHelper.js
■ ■ ■
■ ■ ■■■ jobs/                   # BullMQ queues, Redis-backed
■ ■ ■ ■■■ queue.js              # queue + connection setup
■ ■ ■ ■■■ billGeneration.job.js  # monthly auto bill generation
■ ■ ■ ■■■ notification.job.js    # email/push dispatch worker
■ ■ ■ ■■■ announcementReminder.job.js
■ ■ ■
■ ■ ■■■ docs/
■ ■ ■ ■■■ swagger.yaml
■ ■ ■
■ ■ ■■■ tests/
■ ■ ■ ■■■ unit/
■ ■ ■ ■■■ integration/
■ ■ ■
■ ■ ■■■ app.js                  # express app, middleware wiring
■ ■ ■ ■■■ server.js             # http server bootstrap + DB/Redis connect
■ ■ ■
■ ■■■ uploads/
■ ■ ■■■ complaints/            # local dev storage (swap for S3 in prod)
■ ■■■ .env
■ ■■■ .env.example
■ ■■■ package.json
■ ■■■ README.md
■
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ api/                   # one axios module per resource
■ ■ ■■■ components/            # shared UI (Table, Modal, Badge...)
■ ■ ■■■ pages/
■ ■ ■ ■■■ admin/               # AdminDashboard, ManageResidents, Billing...
■ ■ ■ ■■■ committee/          # CommitteeDashboard, AssignedComplaints...
■ ■ ■ ■■■ resident/           # MyProfile, MyBills, RaiseComplaint...
■ ■ ■ ■■■ context/            # AuthContext, RoleContext
■ ■ ■ ■■■ hooks/              # useAuth, usePagination, useFetch
■ ■ ■ ■■■ routes/             # ProtectedRoute.jsx (role-aware)
■ ■ ■ ■■■ utils/
■ ■ ■ ■■■ App.jsx
■ ■■■ package.json
■
■■■ docs/
■■■ ER-diagram.png
■■■ API-collection.postman_collection.json
■■■ architecture.md

```

Notice that **models**, **controllers**, **services** and **routes** mirror each other one-to-one per resource. This is what a reviewer scans for first under "Project Structure" -- if they can guess where a piece of logic lives just from the module name, that's a pass on this criterion.

4. Database Design — Tables, Columns & Relationships

16 tables, normalized to 3NF. Every foreign key points at a real parent row (no duplicated data), which is exactly what "Database Design" and "database normalization" in the assessment are checking for. PostgreSQL types are shown; adjust naming slightly if you use Prisma/Sequelize conventions.

roles <i>Lookup table for the 3 user roles</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
name	VARCHAR(50) UNIQUE NOT NULL	'admin' 'committee_member' 'resident'
description	TEXT	Optional label shown in admin UI
created_at	TIMESTAMP DEFAULT NOW()	
Relations: Parent of users (1:N via users.role_id).		

societies <i>One row per residential society (supports multi-society/multi-tenant use)</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
name	VARCHAR(150) NOT NULL	
address	TEXT	
city / state / pincode	VARCHAR	Split fields for filtering/search
registration_number	VARCHAR(100)	Society's legal registration no.
created_at	TIMESTAMP DEFAULT NOW()	
Relations: Parent of users , blocks , announcements .		

users <i>Every login-capable person: admin, committee member, or resident</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
society_id	INTEGER REFERENCES societies(id)	
role_id	INTEGER REFERENCES roles(id)	
name	VARCHAR(100) NOT NULL	
email	VARCHAR(150) UNIQUE NOT NULL	Login identifier

Column	Type	Constraints / Notes
password_hash	VARCHAR(255) NOT NULL	bcrypt hash, never plaintext
phone	VARCHAR(15)	
profile_image_url	VARCHAR(255)	
status	VARCHAR(20) DEFAULT 'active'	'active' 'inactive' 'suspended'
created_at / updated_at	TIMESTAMP DEFAULT NOW()	
Relations: Child of roles & societies . Parent of resident_profiles, committee_members, complaints, service_requests, payments, announcements, audit_logs, notifications.		

blocks <i>A tower / wing within a society, e.g. 'Tower A'</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
society_id	INTEGER REFERENCES societies(id)	
block_name	VARCHAR(50) NOT NULL	
Relations: Child of societies . Parent of flats .		

flats <i>An individual residential unit</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
block_id	INTEGER REFERENCES blocks(id)	
flat_number	VARCHAR(20) NOT NULL	
floor	INTEGER	
area_sqft	NUMERIC(8,2)	
is_occupied	BOOLEAN DEFAULT false	
	UNIQUE(block_id, flat_number)	Prevents duplicate flat numbers per block
Relations: Child of blocks . Parent of resident_profiles, maintenance_bills, complaints, service_requests.		

resident_profiles <i>Extra profile fields specific to resident-role users</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
user_id	INTEGER UNIQUE REFERENCES users(id)	1:1 with users
flat_id	INTEGER REFERENCES flats(id)	

Column	Type	Constraints / Notes
is_owner	BOOLEAN DEFAULT true	false = tenant
move_in_date	DATE	
family_members_count	INTEGER DEFAULT 1	
emergency_contact	VARCHAR(15)	
Relations: 1:1 child of users . N:1 child of flats (a flat can have several residents over time).		

committee_members <i>Extra profile fields specific to committee-role users</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
user_id	INTEGER UNIQUE REFERENCES users(id)	1:1 with users
designation	VARCHAR(50)	'President' 'Secretary' 'Treasurer'
term_start / term_end	DATE	
Relations: 1:1 child of users .		

maintenance_bills <i>One bill per flat per billing period</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
flat_id	INTEGER REFERENCES flats(id)	
billing_month	INTEGER NOT NULL	1-12
billing_year	INTEGER NOT NULL	
amount	NUMERIC(10,2) NOT NULL	
due_date	DATE NOT NULL	
status	VARCHAR(20) DEFAULT 'pending'	'pending' 'paid' 'partially_paid' 'overdue'
generated_by	INTEGER REFERENCES users(id)	Admin who generated it
created_at	TIMESTAMP DEFAULT NOW()	
	UNIQUE(flat_id, billing_month, billing_year)	One bill per flat per month
Relations: Child of flats and users (generated_by). Parent of payments .		

payments <i>Payment record(s) against a bill — supports partial payments</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key

Column	Type	Constraints / Notes
bill_id	INTEGER REFERENCES maintenance_bills(id)	
paid_by	INTEGER REFERENCES users(id)	Resident who paid
amount_paid	NUMERIC(10,2) NOT NULL	
payment_mode	VARCHAR(20)	'cash' 'upi' 'card' 'netbanking'
transaction_ref	VARCHAR(100)	Gateway/UPI transaction id
payment_date	TIMESTAMP DEFAULT NOW()	
status	VARCHAR(20) DEFAULT 'success'	'success' 'failed' 'pending'
Relations: Child of maintenance_bills and users .		

complaints <i>Resident-raised issues tracked to resolution</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
raised_by	INTEGER REFERENCES users(id)	
flat_id	INTEGER REFERENCES flats(id)	
category	VARCHAR(50)	'plumbing' 'electrical' 'security' 'cleanliness' 'other'
title	VARCHAR(150) NOT NULL	
description	TEXT	
priority	VARCHAR(20) DEFAULT 'medium'	'low' 'medium' 'high'
status	VARCHAR(20) DEFAULT 'open'	'open' 'in_progress' 'resolved' 'closed'
assigned_to	INTEGER REFERENCES users(id) NULL	Committee member / admin
created_at / updated_at / resolved_at	TIMESTAMP	
Relations: Child of users (raised_by, assigned_to) and flats . Parent of complaint_images, complaint_status_logs.		

complaint_images <i>One or more photos attached to a complaint</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
complaint_id	INTEGER REFERENCES complaints(id)	

Column	Type	Constraints / Notes
image_url	VARCHAR(255) NOT NULL	Local path or S3/Cloudinary URL
uploaded_at	TIMESTAMP DEFAULT NOW()	
Relations: Child of complaints .		

complaint_status_logs <i>Audit trail of every status change on a complaint</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
complaint_id	INTEGER REFERENCES complaints(id)	
updated_by	INTEGER REFERENCES users(id)	
old_status / new_status	VARCHAR(20)	
remark	TEXT	Optional note from the person updating it
created_at	TIMESTAMP DEFAULT NOW()	
Relations: Child of complaints and users .		

service_requests <i>Resident requests for society-arranged services</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
requested_by	INTEGER REFERENCES users(id)	
flat_id	INTEGER REFERENCES flats(id)	
service_type	VARCHAR(50)	'plumber' 'electrician' 'cleaning' 'pest_control' 'other'
description	TEXT	
preferred_date	DATE	
status	VARCHAR(20) DEFAULT 'pending'	'pending' 'approved' 'rejected' 'completed'
approved_by	INTEGER REFERENCES users(id) NULL	
created_at / updated_at	TIMESTAMP	
Relations: Child of users (requested_by, approved_by) and flats .		

announcements <i>Society-wide or audience-targeted notices</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key

Column	Type	Constraints / Notes
society_id	INTEGER REFERENCES societies(id)	
posted_by	INTEGER REFERENCES users(id)	
title	VARCHAR(150) NOT NULL	
content	TEXT NOT NULL	
target_audience	VARCHAR(20) DEFAULT 'all'	'all' 'residents' 'committee'
is_pinned	BOOLEAN DEFAULT false	
created_at / expires_at	TIMESTAMP	
Relations: Child of societies and users .		

audit_logs <i>Immutable trail of sensitive actions across the system</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
user_id	INTEGER REFERENCES users(id)	Who performed the action
action	VARCHAR(100)	e.g. 'CREATE_BILL', 'UPDATE_COMPLAINT_STATUS'
entity_type / entity_id	VARCHAR(50) / INTEGER	Which row was affected
old_value / new_value	JSONB	Before/after snapshot
ip_address	VARCHAR(45)	
created_at	TIMESTAMP DEFAULT NOW()	
Relations: Child of users . Written automatically by auditLogger.middleware.js.		

notifications <i>In-app notification feed per user</i>		
Column	Type	Constraints / Notes
id	SERIAL PK	Primary key
user_id	INTEGER REFERENCES users(id)	
title / message	VARCHAR(150) / TEXT	
type	VARCHAR(30)	'bill' 'complaint' 'announcement' 'service_request'
is_read	BOOLEAN DEFAULT false	
created_at	TIMESTAMP DEFAULT NOW()	
Relations: Child of users . Populated by notification.service.js, typically from a background job.		

5. Entity Relationship Summary

A flat list of every foreign-key relationship in the schema, in the order you'd draw them onto an ER diagram. Recreate this visually in [draw.io](#) / [dbdiagram.io](#) for the optional "Database Schema" submission artifact.

Parent Table	Child Table	Type	FK Column
roles	users	1:N	users.role_id
societies	users	1:N	users.society_id
societies	blocks	1:N	blocks.society_id
societies	announcements	1:N	announcements.society_id
blocks	flats	1:N	flats.block_id
users	resident_profiles	1:1	resident_profiles.user_id
flats	resident_profiles	1:N	resident_profiles.flat_id
users	committee_members	1:1	committee_members.user_id
flats	maintenance_bills	1:N	maintenance_bills.flat_id
users	maintenance_bills	1:N (generated_by)	maintenance_bills.generated_by
maintenance_bills	payments	1:N	payments.bill_id
users	payments	1:N (paid_by)	payments.paid_by
users	complaints	1:N (raised_by)	complaints.raised_by
flats	complaints	1:N	complaints.flat_id
users	complaints	1:N (assigned_to)	complaints.assigned_to
complaints	complaint_images	1:N	complaint_images.complaint_id
complaints	complaint_status_logs	1:N	complaint_status_logs.complaint_id
users	service_requests	1:N (requested_by)	service_requests.requested_by
flats	service_requests	1:N	service_requests.flat_id
users	service_requests	1:N (approved_by)	service_requests.approved_by
users	announcements	1:N (posted_by)	announcements.posted_by
users	audit_logs	1:N	audit_logs.user_id
users	notifications	1:N	notifications.user_id

Reading tip: *users* sits at the centre of the diagram — almost everything eventually traces back to it, which is exactly what you'd expect from a system built around role-based access. *flats* is the second hub, connecting the physical-society side (blocks, societies) to the operational side (bills, complaints, service requests).

6. Redis Usage Plan — What, Where & Why

Redis is not in the assessment's required stack, so nothing breaks if you skip it. But it's the single highest-leverage addition you can make for very little code, because it maps directly onto things the evaluation criteria explicitly reward: **Security Practices** (token revocation, rate limiting), **optimized workflows** (caching, async jobs), and **Overall Implementation** (shows you know when a relational database is the wrong tool for a job). Below is exactly where each use case plugs into the folder structure from Section 3.

Use Case	Redis Key Pattern	TTL	Implemented In
Refresh token store (rotate on every use)	refresh:{userId}:{tokenId}	7 days	auth.service.js, config/redis.config.js
Access-token blacklist (on logout)	blacklist:{titi}	remaining token life	auth.middleware.js
Login rate limiting (brute-force protection)	ratelimit:login:{ip}	15 min window	middlewares/rateLimiter.middleware.js
Password-reset OTP	otp:{userId}	5 min	auth.service.js
Dashboard stats cache (admin summary counts)	dashboard:summary:{societyId}	5 min, invalidated on writes	dashboard.controller.js, cache.service.js
Announcements list cache (read-heavy, low-change data)	announcements:{societyId}	10 min, invalidated on new post	announcement.service.js
Payment idempotency key (prevents duplicate entries on retry)	idempotency:{clientId}	24 hr	payment.service.js
Background job queues (BullMQ)	queue:bill-generation, queue:notifications	persistent	jobs/ folder
Real-time notice push (optional — Socket.io scaling)	pub/sub channel per society	n/a	server.js (socket adapter)

6.1 Why each one earns its place

- **Refresh tokens in Redis, not Postgres:** revoking a session (logout, "log out of all devices") becomes a single DEL instead of an UPDATE plus an index scan, and expiry is handled natively by TTL instead of a cron job checking timestamps.
- **Rate limiting:** Postgres is the wrong tool for a counter that resets every few minutes and is checked on every login attempt — Redis's INCR + EXPIRE is built exactly for this.
- **Caching dashboard/announcement reads:** these are read far more often than they change. Serving them from Redis instead of re-running aggregation queries on every page load is the textbook definition of "optimized workflow".
- **Idempotency keys on payments:** a resident's payment request might get retried by a flaky network connection — without a short-lived idempotency guard you risk double-charging or double-recording a payment.
- **BullMQ queues:** generating maintenance bills for every flat on the 1st of the month, or sending a batch of notification emails, should never block an HTTP response — Redis is what BullMQ uses under the hood to persist and process jobs.

Where the client lives in code: config/redis.config.js exports a single ioredis instance. services/cache.service.js wraps it with simple get(key) / set(key, value, ttl) / del(key)

helpers (JSON-serializing automatically) so no other file talks to Redis directly — same pattern as your db.config.js / model layer split.

7. Functional Module → Code Mapping

Use this as your build order. Each row is one functional module from the assessment brief, mapped to the exact files it touches in the folder structure from Section 3.

Module	Controller / Routes	Service(s)	Tables Used	Key Middleware
User Authentication	auth.controller.js auth.routes.js	auth.service.js	users, roles	rateLimiter, validate
Resident Management	resident.controller.js resident.routes.js	user.service.js	users, resident_profiles, flats	auth, rbac(admin)
Committee Member Mgmt	committee.controller.js committee.routes.js	user.service.js	users, committee_members	auth, rbac(admin)
Announcements & Notices	announcement.controller.js announcement.routes.js	announcement.service.js	announcements	auth, rbac(admin/committee), validate
Complaint Management	complaint.controller.js complaint.routes.js	complaint.service.js	complaints, complaint_images, complaint_status_logs	auth, upload (multer), rbac
Service Request Mgmt	serviceRequest.controller.js serviceRequest.routes.js	serviceRequest.service.js	service_requests	auth, rbac
Maintenance Billing	maintenanceBill.controller.js maintenanceBill.routes.js	maintenanceBill.service.js jobs/billGeneration.job.js	maintenance_bills, flats	auth, rbac(admin), validate
Payment Records	payment.controller.js payment.routes.js	payment.service.js	payments, maintenance_bills	auth, idempotency (Redis)
Dashboard & Reports	dashboard.controller.js dashboard.routes.js	aggregation across services cache.service.js	all tables (read-only aggregates)	auth, rbac(admin/committee)
Role-Based Access Control	applied in every routes/*.js file	—	roles, users	rbac.middleware.js (cross-cutting)
Audit Logs	auditLog.controller.js auditLog.routes.js	auditLog.service.js	audit_logs	auditLogger.middleware.js (global), rbac(admin)

8. REST API Endpoint Design

All list endpoints accept ?page, ?limit, ?search and ?status query params where relevant, satisfying the "pagination, filtering, and search" requirement. All endpoints below sit under a common /api/v1 prefix.

8.1 Authentication

Method	Endpoint	Access	Purpose
POST	/auth/register	Admin only (creates resident/ committee logins)	Create a new user
POST	/auth/login	Public	Email + password → access + refresh token
POST	/auth/refresh	Public (valid refresh token)	Issue new access token
POST	/auth/logout	Authenticated	Revoke refresh token, blacklist access token
POST	/auth/forgot-password	Public	Sends OTP (stored in Redis)
POST	/auth/reset-password	Public (valid OTP)	Set new password

8.2 Residents & Committee

Method	Endpoint	Access	Purpose
GET	/residents	Admin, Committee	Paginated resident directory
GET	/residents/:id	Admin, Committee, Self	Single resident profile
POST	/residents	Admin	Onboard a new resident
PUT	/residents/:id	Admin, Self (own profile)	Update resident profile
DELETE	/residents/:id	Admin	Deactivate resident
GET	/committee	Admin	List committee members
POST	/committee	Admin	Appoint a committee member
PUT	/committee/:id	Admin	Update designation / term

8.3 Blocks, Flats & Announcements

Method	Endpoint	Access	Purpose
GET / POST	/blocks	Admin	List / create blocks
GET / POST	/flats	Admin	List / create flats

Method	Endpoint	Access	Purpose
PUT	/flats/:id	Admin	Update occupancy / details
GET	/announcements	All roles	List notices (cached in Redis)
POST	/announcements	Admin, Committee (if permitted)	Publish a notice
PUT	/announcements/:id	Admin, author	Edit a notice
DELETE	/announcements/:id	Admin	Remove a notice

8.4 Complaints & Service Requests

Method	Endpoint	Access	Purpose
GET	/complaints	Admin, Committee (assigned), Resident (own)	List complaints, filter by status/category
GET	/complaints/:id	Admin, Committee, Owner	Complaint detail + images + status trail
POST	/complaints	Resident	multipart/form-data with image upload
PUT	/complaints/:id/status	Admin, assigned Committee member	Update status, writes to complaint_status_logs
GET	/service-requests	Admin, Committee, Resident (own)	List service requests
POST	/service-requests	Resident	Raise a new request
PUT	/service-requests/:id/status	Admin, Committee	Approve / reject / complete

8.5 Billing, Payments, Dashboard & Audit

Method	Endpoint	Access	Purpose
GET	/bills	Admin, Resident (own)	List bills, filter by status/month
POST	/bills	Admin	Generate a bill (also triggered by billGeneration.job.js)
GET	/bills/flat/:flatId	Admin, Resident (own flat)	Bill history for a flat
GET	/payments	Admin, Resident (own)	Payment history
POST	/payments	Resident	Record a payment (idempotency-key header supported)
GET	/dashboard/summary	Admin, Committee	Aggregated counts (cached, 5 min TTL)

Method	Endpoint	Access	Purpose
GET	/dashboard/reports/complaints	Admin, Committee	Complaint trend report
GET	/audit-logs	Admin only	Paginated, filterable audit trail

9. Role-Based Access Control (RBAC) Matrix

Enforced by `rbac.middleware.js`, e.g. `router.post('/bills', auth, allowRoles('admin'), createBill)`. "Own only" means the service layer additionally filters by the requester's own `user_id` / `flat_id` — role alone isn't enough to scope those rows.

Module	Admin	Committee Member	Resident
Resident Management	Full CRUD	View only	View / edit own profile
Committee Management	Full CRUD	View only (self edit)	No access
Announcements	Full CRUD	Create / update (if permitted), view	View only
Complaints	View all, assign, update status, delete	View assigned, update status	Create own, view own, upload images
Service Requests	Approve / reject / view all	Process assigned, update status	Create own, view own
Maintenance Billing	Full CRUD, generate bills	View only	View own bills
Payments	Record / view all	View only	View own, make payment
Dashboard & Reports	Full access	Limited (assigned-scope data)	Personal summary only
Audit Logs	Full access	No access	No access

10. Security & Validation Checklist

10.1 Authentication & Session Security

- Passwords hashed with bcrypt (10-12 salt rounds), never stored or logged in plaintext.
- Short-lived access tokens (~15 min) + longer-lived refresh tokens (~7 days), refresh tokens rotated and stored in Redis so any single one can be revoked instantly.
- Login endpoint rate-limited per IP and per account (Redis-backed) to blunt brute-force attempts.
- Generic "invalid credentials" error on login failure — never reveal whether the email exists.

10.2 Input Validation & Data Handling

- Every mutating endpoint validated with Joi/Zod schemas before touching a controller (validate.middleware.js).
- Parameterized queries / ORM query builders only — no string-concatenated SQL, anywhere.
- File uploads restricted by MIME type and size (e.g. image/jpeg, image/png only, 5MB max) in multer.config.js.
- All user-supplied strings rendered back to the frontend are escaped to prevent stored XSS in complaint descriptions, announcement content, etc.

10.3 API & Infrastructure Hardening

- helmet() for secure HTTP headers, cors() with an explicit origin whitelist (never '*' once a frontend URL is known).
- Centralised errorHandler.middleware.js so stack traces / internal errors never leak to API responses.
- Environment secrets (DB credentials, JWT secret, Redis URL) only in .env, never committed — .env.example documents the required keys with placeholder values.
- Sensitive mutations (bill generation, complaint status change, user deactivation) written to audit_logs automatically via auditLogger.middleware.js.

11. Testing & Documentation Plan

11.1 Testing (Jest + Supertest)

- Unit tests per service file (e.g. `maintenanceBill.service.js`: bill amount calculation, duplicate-bill prevention).
- Integration tests per route group hitting a test database (e.g. `complaint.routes.js`: `create` → `assign` → `resolve` lifecycle).
- RBAC tests: assert a resident token gets 403 on an admin-only route, and vice versa.
- Aim for coverage on the riskiest logic first: auth, billing calculations, RBAC — not 100% coverage for its own sake.

11.2 API Documentation

- Swagger/OpenAPI spec generated from JSDoc comments above each route (swagger-jsdoc + swagger-ui-express), served at `/api-docs`.
- Export a matching Postman collection (`docs/API-collection.postman_collection.json`) with example requests for every endpoint in Section 8, including auth headers.

11.3 README.md Contents

- Project overview and tech stack (link back to Section 2).
- Setup instructions: `clone` → `npm install` → `configure .env` → `run migrations` → `seed roles` → `npm run dev`.
- Environment variables table (name, description, example value — never real secrets).
- Database setup: how to create the Postgres DB, run migrations, and (optionally) start Redis via Docker.
- API documentation link (Swagger UI URL) and Postman collection location.
- Screenshots of the Admin, Committee and Resident dashboards (optional but recommended by the brief).

12. Submission Checklist (Don't Forget)

Straight from the assessment brief's "Submission Requirements" and "GitHub Repository Guidelines" sections — tick each before you submit.

Item	Status / Note
Public GitHub repository created	
Regular commits with meaningful messages (not one giant final commit)	
README.md: setup instructions	
README.md: technology stack used	
README.md: database setup steps	
README.md: environment variables table	
README.md: API documentation (link or embedded)	
README.md: screenshots (optional, recommended)	
Database schema exported (SQL dump or ER diagram image)	
Postman collection or Swagger spec included	
Deployment link (optional)	
RBAC verified for all 3 roles on at least the core flows	
Complaint image upload tested end-to-end	
Maintenance bill → payment flow tested end-to-end	

That's the full blueprint. Build Section 4 (database) and Section 3 (folders) first, then work module by module through Section 7 — by the time you reach the last row of the checklist above, the assessment is done.